

ON THIS PAGE

Prerequisites Quick Start Project Structure Add a Page Object Add a Test Class

BasePage Methods React Patterns Running Tests Modules Reference

TEST AUTOMATION FRAMEWORK

selenium-java-framework

A Page Object Model framework built on Selenium 4, Java 17, and TestNG for end-to-end testing of SauceDemo. Clone, extend, and run — no additional setup required.

● 54 tests passing

● 8 modules

● Selenium 4.18.1

● Java 17

● TestNG 7.9.0

Prerequisites

Install these tools before cloning. Versions must match to avoid ChromeDriver compatibility errors.

Java JDK

17 or higher

Set JAVA_HOME in system environment variables

Apache Maven

3.9.x

Add Maven bin/ to your PATH

Google Chrome

150.x (must match driver)

Check your version: chrome://settings/help

ChromeDriver

150.x (auto-managed)

WebDriverManager downloads it automatically — no manual install

Quick Start

1 Clone the repository

```
$ git clone https://github.com/NidhiSingh-1902/selenium-java-framework.git
$ cd selenium-java-framework
```

2 Run the full suite

This compiles the project, launches Chrome, runs all 54 tests, and generates a report.

```
$ mvn test

# Run a single module only
$ mvn test -Dtest=CartTest

# Run multiple modules
$ mvn test -Dtest=CartTest,CheckoutStep1Test
```

3 Check results

Test reports are written to `target/surefire-reports/` after each run. Open any `.txt` file for a plain-text summary or the `.xml` files for CI integration.

Project Structure

```
selenium-java-framework/ |— src/main/java/com/ | |— framework/ | | |— base/
BaseTest.java — @BeforeMethod / @AfterMethod, driver lifecycle | | |— pages/
BasePage.java — click, type, typeReact, jsClick, getText... | | |— utils/
WaitUtils.java — waitForVisible, waitForUrlContains... | | |— pages/ | |—
LoginPage.java | |— ProductsPage.java | |— ProductDetailPage.java | |—
CartPage.java | |— CheckoutStep1Page.java | |— CheckoutStep2Page.java | |—
OrderCompletePage.java | |— BurgerMenuPage.java | |— src/test/java/com/tests/ |
|— LoginTest.java — 6 tests | |— ProductsTest.java — 10 tests | |—
ProductDetailTest.java — 7 tests | |— CartTest.java — 8 tests | |—
CheckoutStep1Test.java — 7 tests | |— CheckoutStep2Test.java — 6 tests | |—
OrderCompleteTest.java — 5 tests | |— MenuTest.java — 5 tests | |—
src/test/resources/ |— testng.xml — suite config, parallel="methods" thread-
count="2"
```

Add a New Page Object

Every page of the application gets its own class. The pattern is always the same: extend `BasePage`, declare locators with `@FindBy`, write action methods.

Rule: Never put raw Selenium calls in test classes. All browser interactions belong in page objects.

```
package com.pages;

import com.framework.pages.BasePage;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;

public class MyNewPage extends BasePage {

    // 1. Declare locators with @FindBy - PageFactory wires them up automatically
    @FindBy(id = "page-title")
    private WebElement pageTitle;

    @FindBy(css = ".submit-btn")
    private WebElement submitButton;

    // 2. Constructor - always call super(driver)
    public MyNewPage(WebDriver driver) {
        super(driver);
    }

    // 3. Page state
    public boolean isPageLoaded() {
        return isDisplayed(pageTitle)
            && driver.getCurrentUrl().contains("my-page");
    }

    // 4. Action methods - use inherited helpers, not raw Selenium
    public void clickSubmit() {
        wait.waitForVisible(submitButton);
        jsClick(submitButton);
        wait.waitForUrlContains("next-page");
    }
}
```

Register it in testng.xml

Add a new `<test>` block for the corresponding test class:

```
<!-- src/test/resources/testng.xml -->
<test name="My New Page Tests">
    <classes>
        <class name="com.tests.MyNewPageTest"/>
    </classes>
</test>
```

Add a New Test Class

Test classes extend `BaseTest` (which manages the browser lifecycle) and follow the **Arrange** → **Act** → **Assert** pattern.

```
package com.tests;

import com.framework.base.BaseTest;
import com.pages.LoginPage;
import com.pages.MyNewPage;
import org.testng.Assert;
import org.testng.annotations.Test;

public class MyNewPageTest extends BaseTest {

    // Helper - shared setup steps go here, not in @BeforeMethod
    private MyNewPage loginAndNavigate() {
        LoginPage login = new LoginPage(getDriver());
        login.login("standard_user", "secret_sauce");
        // ... navigate to the page
        return new MyNewPage(getDriver());
    }

    // Test ID format: TC-[MODULE]-[NUMBER]
    @Test(priority = 1, description = "TC-MN-01: Page should load with correct heading")
    public void pageLoadsTest() {
        // Arrange + Act
        MyNewPage page = loginAndNavigate();

        // Assert
```

```
Assert.assertTrue(page.isPageLoaded(), "Page should be loaded");
log.info("TC-MN-01 PASSED");
}
}
```

Test ID naming convention

MODULE	PREFIX	EXAMPLE
Login	TC-LG-	TC-LG-01
Products	TC-PR-	TC-PR-04
Product Detail	TC-PD-	TC-PD-02
Cart	TC-CT-	TC-CT-05
Checkout Step 1	TC-C1-	TC-C1-03
Checkout Step 2	TC-C2-	TC-C2-06
Order Complete	TC-OC-	TC-OC-01
Burger Menu	TC-MN-	TC-MN-05
Your new module	TC-XX-	TC-XX-01

BasePage Methods

All page objects inherit these methods. Use them instead of calling Selenium directly.

METHOD	WHEN TO USE
click(element)	Standard click — waits for element to be clickable first
jsClick(element)	Use this for all React buttons and links. JavaScript click bypasses React's synthetic event system issues
type(element, text)	Clear field and type — for non-React inputs (login form, search boxes)

METHOD	WHEN TO USE
<code>typeReact(element, text)</code>	Use this for React controlled inputs. Sets value via native setter + fires an input event so React's <code>onChange</code> updates internal state
<code>getText(element)</code>	Get visible text — waits for element visibility, returns trimmed string
<code>isDisplayed(element)</code>	Safe visibility check — returns false instead of throwing when element is absent
<code>scrollToElement(element)</code>	Scroll element into viewport — use before interacting with off-screen elements
<code>getPageTitle()</code>	Returns the browser tab title
<code>getCurrentUrl()</code>	Returns the current browser URL

WaitUtils (via `wait.*`)

METHOD	WHEN TO USE
<code>wait.waitForVisible(element)</code>	Wait until element is visible — use before reading text or clicking
<code>wait.waitForClickable(element)</code>	Wait until element is enabled and clickable
<code>wait.waitForUrlContains(fragment)</code>	Wait for navigation — pass a URL fragment like <code>"cart.html"</code>
<code>wait.waitForTextInElement(By, text)</code>	Stale-safe text wait — use after React re-renders a button (e.g. "Add to cart" → "Remove")

| React-Specific Patterns

SauceDemo is built with React. Two common Selenium approaches silently fail on React pages — here is why and what to use instead.

Clicking buttons and links

Problem: `element.click()` triggers a synthetic click that React may not handle. Use `jsClick(element)` for all cart buttons, nav links, and form submits.

```
// X Do not use - misses React event handlers on re-rendered nodes
element.click();

// ✓ Use this - fires a native DOM click React always catches
jsClick(element);
```

Typing into form fields

Problem: `element.clear()` resets the DOM value but does not update React's internal state. The form still sees an empty field when submitted. Use `typeReact()` for all checkout form inputs.

```
// X Do not use for React inputs - React's internal state stays empty
element.clear();
element.sendKeys("John");

// ✓ Use this - sets value via native setter + dispatches 'input' event
typeReact(firstNameField, "John"); // React sees the value correctly
typeReact(lastNameField, "Doe");
typeReact(postalCodeField, "12345");
```

Waiting after React state changes

Problem: After clicking "Add to cart", React re-renders the button node. A stale `WebElement` reference to the old node throws `StaleElementReferenceException`. Use `waitForTextInElement(By, text)` with a locator, not an element reference.

```
// X Stale - the By.id locator-based element reference may be dead after React re-render
wait.until(ExpectedConditions.textToBePresentInElement(button, "Remove"));

// ✓ Safe - re-finds the element by locator on each retry
By locator = By.cssSelector(".inventory_item:nth-child(1) .btn_inventory");
wait.waitForTextInElement(locator, "Remove");
```

Running Tests

COMMAND	WHAT IT RUNS
<code>mvn test</code>	Full suite — all 54 tests via testng.xml
<code>mvn test -Dtest=CartTest</code>	Single test class
<code>mvn test -Dtest=CartTest,CheckoutStep1Test</code>	Multiple specific classes
<code>mvn test -Dtest=CartTest#cartPageLoadsTest</code>	Single test method
<code>mvn clean test</code>	Clean compiled classes first, then run suite

Test reports

After `mvn test`, reports appear in `target/surefire-reports/`. Each class gets its own `.txt` summary. Look for lines marked `FAILURE!` to identify broken tests.

```
# Open the summary for a specific class
$ cat target/surefire-reports/com.tests.CartTest.txt
```

Modules Reference

All 8 modules cover the complete SauceDemo user journey end-to-end.

MODULE	TEST CLASS	TESTS	URL	STATUS
Login	LoginTest	6	/	PASS
Products	ProductsTest	10	/inventory.html	PASS
Product Detail	ProductDetailTest	7	/inventory-item.html	PASS
Cart	CartTest	8	/cart.html	PASS
Checkout Step 1	CheckoutStep1Test	7	/checkout-step-one.html	PASS
Checkout Step 2	CheckoutStep2Test	6	/checkout-step-two.html	PASS
Order Complete	OrderCompleteTest	5	/checkout-complete.html	PASS
Burger Menu	MenuTest	5	overlay (all pages)	PASS

